Team: Daniel Brown, Noah Dougherty, Tyler Baker, William Schoennberger

A. Hierarchy

- We split the program into a few main parts. Each part does one job, then the parts get put together at the end.

1. Get inputs
    - Ask for the time, clock type, and character.

2. Check character
    - Make sure it is allowed. If it is not, ask again. A blank is okay.

3. Change the time if needed
    - For a 12-hour clock, figure out AM or PM and change the hour.

4. Set up the art
    - Use a dictionary for the number patterns, plus the colon and AM/PM letters.

5. Build each row
    - Look up the pattern for each time character and add the right spaces.

6. Print it
    - Print the five rows, adding AM or PM for 12-hour mode.

B. Variables

Name	What it is for
time	The time the user inputs.
cl_type	Whether the user chooses 12- or 24-hour clock time.
pref_char	The character used to draw the numbers.
pref_charog	Keeps the original character so a blank input can be recognized.
ampm	Stores AM or PM for a 12-hour clock.
allowed_chars	The characters the user is allowed to enter.
digit_art	A dictionary with the five-line pattern for each number.
timeprint	A dictionary that stores the five lines that will be printed.

C. Test cases

Some of the test cases we can try after the code is put together are:

Test 1
Input: 9:07, clock type 24, character left blank.
Expected: shows 9:07 in 24-hour format. Each number is drawn using itself.

Test 2
Input: 12:45, clock type 12, character = @.
Expected: shows 12:45 PM, using @ for the digits.

Test 3
Input: 23:59, clock type 12, character = =.
Expected: changes the time to 11:59 PM and uses = for the digits.

Test 4
Input: 06:15, clock type 24, character = a.
Expected: shows 06:15, including the first zero, using a for the digits.

Test 5
Input: 11:00, clock type 12, character left blank.
Expected: shows 11:00 AM and uses the actual numbers for the digit art.

G. Team reflection

1. Describe the difficulty your team had when combining the code at the end. How could your team have specified the design more clearly at the beginning?

Combining the code was difficult because everyone worked on different parts, and we had to make sure they all worked together effectively. Some parts used the same variables, so changing something in one section could affect the other section. We also had to make sure the time conversion and number patterns worked correctly with each other. Next time, we could agree on variable names and how the code would be organized before splitting up the work into parts.

2. Describe any benefits and drawbacks you saw by dividing the coding like this. Can you see reasons why this might be a good idea? How about a bad idea?

Splitting up the work helped because everyone could focus on one part of the program and work at the same time, and not worry about the other parts. This made the assignment easier to manage with a group. However, putting the code together was harder because everyone's section had to work with the others in the group. If people used different variable names or made different assumptions than others, it could cause errors. To make this easier, we should communicate throughout the project and test the full program together.